

Glossary

Term	Definition
Locator	A method used by Playwright to find and interact with webpage elements.
Assertion	A check that verifies whether the actual result matches the expected result.
Automation	Using software to perform testing tasks automatically.
Repository	A storage location where Git keeps project files and their version history.
Staging Area	A Git area where changes are prepared before committing.
.gitignore	A file that tells Git which files or folders should not be tracked.

Technical Learning Report: Week 4 Day 4

Date: September 29, 2026

Intern Name: Shekhar Singh Rawal

Subject: Python Basics, Playwright Automation & Git/GitHub Fundamentals

1. Objective

The objective of Day 4 was to build foundational knowledge of **Python programming, Playwright automation, and Git/GitHub**. The learning focused on Python variables, conditions, loops, data types and functions, followed by Playwright browser automation, navigation, locators and assertions. The session also introduced Git and GitHub as tools for version control and sharing automation code.

2. Technical Breakdown

2.1 Python Fundamentals

- **Variables:** A variable is a name used to store a value. Example: age = 24
- **If/else:** Used to make a decision based on whether a condition is true or false.
- **Modulo %:** Gives the remainder after division. Example: 24 % 2 = 0
- **Nested if:** An if statement inside another if statement.
- **For loop:** Repeats code for each item in a sequence or range.

- **While loop:** Repeats code as long as a condition is true.
- **List:** An ordered collection that can store multiple values. Example: ["shekhar", "rawal", "QA"]
- **Data types:** Tell Python what kind of value something is, such as int, string, list, or tuple.
- **Function:** A reusable block of code that performs a specific task.
- **print():** Displays a value or message on the screen.
- **type():** Tells you what data type a value has.

Example

1. This example uses the modulo operator to determine whether a number is odd or even.

```
age = 24

if age % 2:
    print("Odd Age")
else:
    print("Even Age")
```

2. A function was also created to perform addition:

```
def ADD(a, b):
    c = a + b
    print(a, "+", b, "=", c)
```

```
ADD(4, 5)
```

3. Loops iterating throw list

```
items_loop = ['shekhar', 'rawal', 'qa']
for il in range(len(items_loop)):
    print(items_loop[il])
```

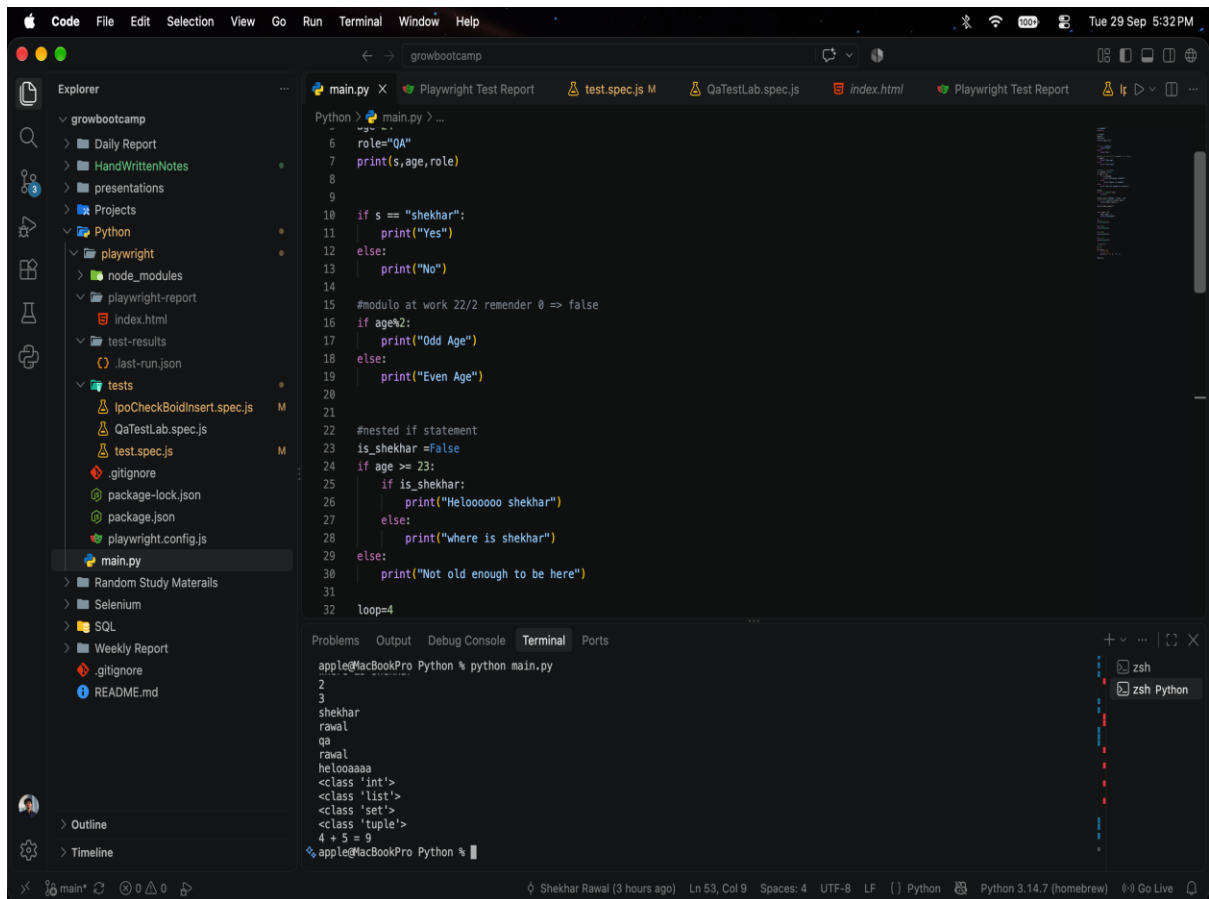


Figure 1: Python Code and Outputs

3. Playwright Automation

Playwright is an automation framework that can control browsers and perform actions such as opening pages, clicking elements, entering data and verifying results.

The Playwright scripts practiced:

- Opening a browser page
- Navigating to URLs
- Locating elements
- Clicking elements
- Entering text
- Reading page titles
- Verifying URLs
- Using assertions

Script	Business Flow Protected	Selector/Method
Website title verification	Ensures the application opens with the expected title	page.title() / toHaveTitle()
QA Test Lab button	Ensures clicking the button navigates to the quote page	CSS locator
CDSC BOID input	Ensures the BOID field accepts user input	XPath locator
Basic assertion tests	Demonstrates value and text assertions	toBe(), toContain(), toBeTruthy()

Example: Page Title Verification

The screenshot shows a VS Code editor with a Playwright test script in `test.spec.js`. The script includes several tests: 'My first Test', 'My Second Test', 'My Third Test', and 'Verify the title of the page'. The 'Verify the title of the page' test navigates to `https://www.shekharrawal.com.np/` and asserts that the page title is 'Siddhanath Multiple Campus (SNSC) | Smart Attendance Management System'.

```

1  const { test, expect } = require('@playwright/test')
2
3  test("My first Test", async function ({ page }) {
4    expect(12).toBe(12)
5  })
6
7  test("My Second Test", async function ({ page }) {
8    expect("shekhar rawal").toContain("she")
9  })
10
11 test("My Third Test", async function ({ page }) {
12   expect("shekhar rawal".includes("rawal")).toBeTruthy()
13 })
14
15 test("Verify the title of the page", async function ({ page }) {
16   await page.goto("https://www.shekharrawal.com.np/")
17   const title = await page.title()
18   console.log(title)
19
20   await expect(page).toHaveTitle("Siddhanath Multiple Campus (SNSC) | Smart Attendance Management System")
21 })
22
23

```

Example: Button Click

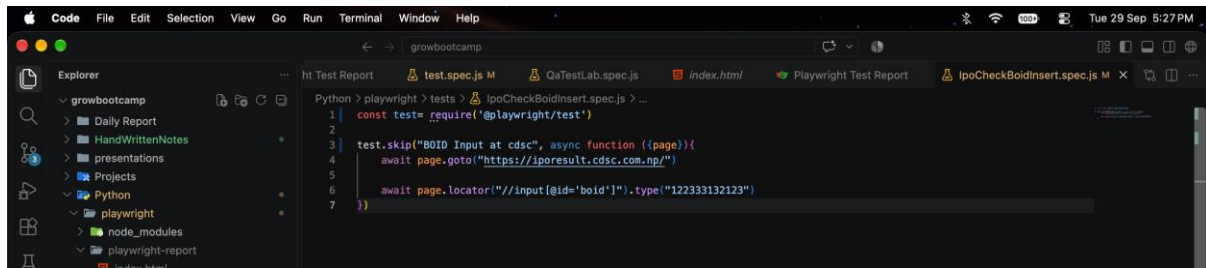
The screenshot shows a VS Code editor with a Playwright test script in `QaTestLab.spec.js`. The script includes a test 'Button Click on QA Test Lab' that navigates to `https://qatestlab.com/`, clicks on a button with the CSS selector `'a.qodef-btn.btn-orange.ml20'`, and asserts that the page URL is `/request-a-quote/`.

```

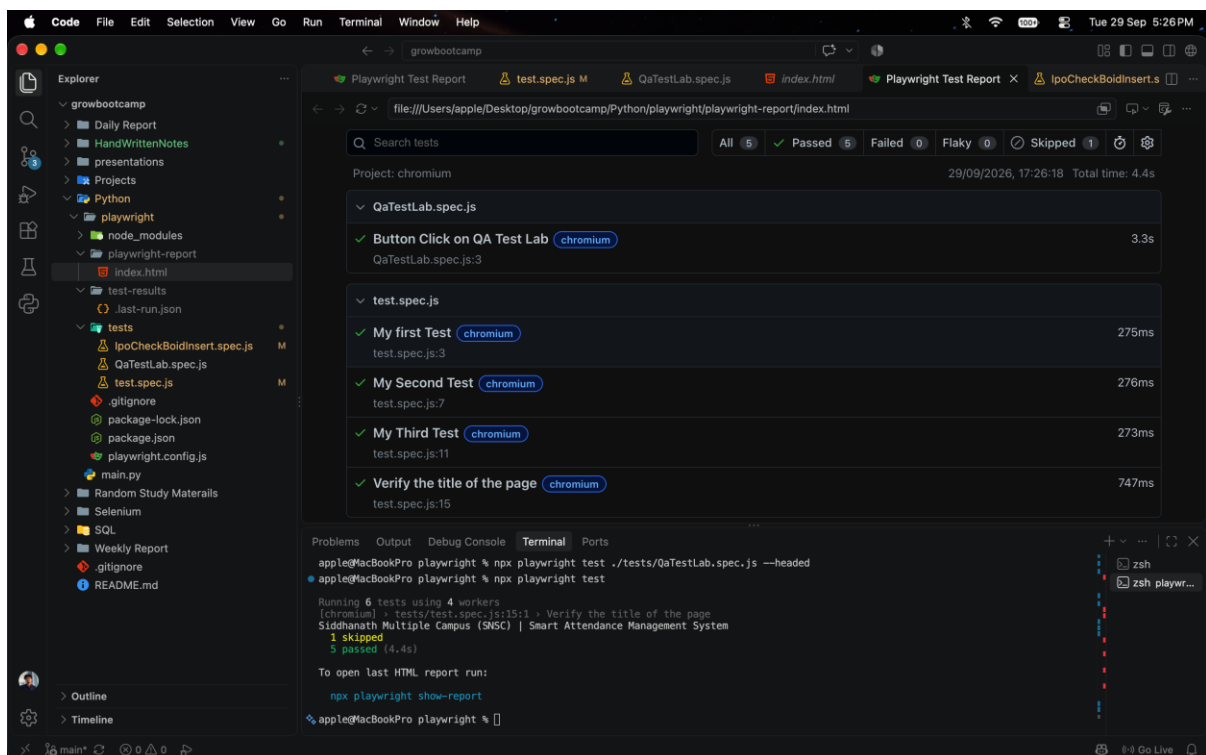
1  const { test, expect } = require('@playwright/test');
2
3  test('Button Click on QA Test Lab', async ({ page }) => {
4    await page.goto('https://qatestlab.com/');
5
6    await page.locator('a.qodef-btn.btn-orange.ml20').click();
7
8    await expect(page).toHaveURL(/request-a-quote/);
9  });

```

Example: Text Insertion



Test Outcomes



4. Understanding Playwright Assertions

Assertions are used to verify whether the actual result matches the expected result.

Examples practiced:

`expect(12).toBe(12);`

Checks whether two values are equal.

`expect("shekhar rawal").toContain("she");`

Checks whether text contains the expected value.

`await expect(page).toHaveTitle("Expected Title");`

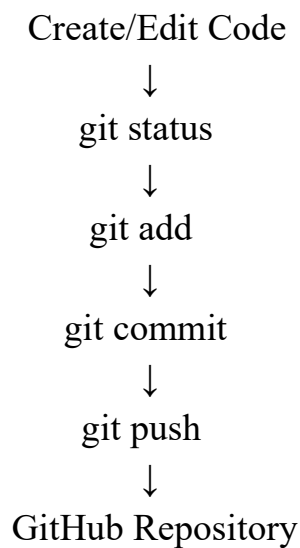
Checks whether the webpage has the expected title.
This demonstrated an important automation principle:
Action → Expected Result → Assertion

5. Git & GitHub Fundamentals

Git was introduced as a **version control system** used to track changes in code.

Command	Purpose
git init	Creates a Git repository
git status	Shows changed/untracked files
git add	Adds changes to the staging area
git commit	Saves a version of the changes
git push	Uploads commits to a remote repository
git pull	Retrieves changes from a remote repository
git clone	Copies an existing repository
git log	Shows previous commits

Basic Git Workflow



.gitignore

A .gitignore file is used to prevent unnecessary or sensitive files from being committed to Git.

For example, generated cache files or local environment files can be excluded from the repository.

6. Practical Work

The provided practical code demonstrates:

- Python variables and basic operations.
- Conditional statements and nested conditions.
- for and while loops.
- Lists and basic Python data types.
- Python functions.
- Playwright assertions.
- Website navigation.
- Page title verification.
- Button clicking.
- URL verification.
- Input field interaction.
- CSS and XPath locator usage.

The assignment required **10 basic Playwright scripts**. The provided evidence contains several Playwright scripts, but does not establish that all 10 were completed. Similarly, no GitHub repository link was provided, so the repository upload cannot be marked as evidenced.

7. AI-Assisted Learning

AI was included as a learning aid for understanding Python, Playwright and Git concepts.

Example prompts included:

```
“from playwright.sync_api import Page, expect
```

```
def test_saucedemo_login(page: Page):  
    # Open SauceDemo  
    page.goto("https://www.saucedemo.com/")  
  
    # Login  
    page.get_by_placeholder("Username").fill("standard_user")  
    page.get_by_placeholder("Password").fill("secret_sauce")  
    page.get_by_role("button", name="Login").click()  
  
    # Verify dashboard/products title  
    expect(page).to_have_title("Swag Labs")  
”
```

8. Key Findings & Best Practices

- Python fundamentals provide the programming base required for automation.
- Playwright can automate browser actions and verify expected results.
- Good locators make automation scripts more reliable.
- Assertions are necessary to determine whether a test actually passes.
- Git helps track changes to automation code.
- GitHub allows teams to store and share Git repositories.
- .gitignore helps prevent unnecessary files from being uploaded.
- Automation code should be reviewed rather than blindly copied from AI.

9. Conclusion

Day 4 introduced the foundations required for test automation by combining **Python programming, Playwright browser automation, and Git/GitHub version control**. Practical scripts demonstrated navigation, element interaction, input handling, locators and assertions. Git concepts provided an understanding of how automation code can be versioned and shared in a team environment.